Content Delivery Networks (CDN)

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

Learning Objectives

- Explain what a CDN is and why it exists
- Distinguish between an origin server and an edge server
- Describe the cache hit/miss lifecycle and how TTL governs it
- Explain how Anycast differs from Unicast routing
- Compare AWS CloudFront and GCP Cloud CDN at a high level
- Identify when to use — and when to avoid — a CDN

The Problem

The Problem CDNs Solve

- The latency problem: every millisecond costs users
- Single origin server bottleneck: traffic, geography, hardware failure
- Real-world motivation: Netflix, Amazon, Facebook all serve traffic through CDNs
- CDN definition: a geographically distributed network of proxy servers that cache content closer to users

Single origin vs. CDN

```
WITHOUT CDN
──────────────────────────────────────────────────────────────

 [User in Tokyo] ──────────────────────────► [Origin: US-East]
 [User in London] ─────────────────────────► [Origin: US-East]
 [User in São Paulo] ──────────────────────► [Origin: US-East]

 All users → long round-trip → slow load times

WITH CDN
──────────────────────────────────────────────────────────

 [User in Tokyo]     ──► [Edge: Tokyo]     ──(cache hit)──► response
 [User in London]    ──► [Edge: London]    ──(cache hit)──► response
 [User in São Paulo] ──► [Edge: São Paulo] ──(cache hit)──► response

 Cache miss only? ──► [Edge] ──► [Origin] ──► fills cache
```

Core Architecture

Origin Server vs. Edge Server

- Origin server: the authoritative source of content; handles dynamic requests
- Edge server (PoP — Point of Presence): caches and serves static content from the nearest location
- The edge server intercepts requests before they reach the origin
- Static content (images, CSS, JS, videos) → cacheable at edge
- Dynamic content (login, checkout, user data via POST) → must reach origin

```
REQUEST FLOW
──────────────────────────────────────────────────────────────────────
Client ──► Edge Server ──(cache HIT?)──► YES ──► return cached file
                        │
                        └──(cache MISS)──► Origin Server
                                                  │
                                            fetch + cache
                                                  │
                                            ◄── return to client
                                            (now cached at edge)
```

Cache Hit, Cache Miss, and TTL

- **Cache hit**: content found at edge → served immediately, no origin trip
- **Cache miss**: content not found or expired → edge fetches from origin
- **TTL (Time-to-Live)**: how long a cached object is considered valid before the edge must re-fetch
  - Short TTL → fresher content, more origin load
  - Long TTL → more cache hits, stale risk
- HTTP Cache-Control header set by the application controls TTL behavior
  - Example: Cache-Control: max-age=86400 tells the CDN to cache for 24 hours

Anycast vs. Unicast

- **Unicast**: one IP address → one specific server (traditional routing)
- **Anycast**: one IP address → multiple servers; router sends traffic to the *nearest* one
- CDNs use Anycast so that the same IP resolves to different edge nodes for different users
- Benefits: lower latency, built-in load balancing, resilience to DDoS attacks

```
UNICAST
──────────────────────────────────────────
 IP: 93.184.216.34  ──►  ONE fixed server
 (All global traffic hits same endpoint)

ANYCAST
──────────────────────────────────────────
 IP: 205.251.194.72  ──►  PoP nearest to user
                     ──►  Tokyo for Japanese users
                     ──►  Frankfurt for German users
                     ──►  Ashburn for US East users
```

CDN Benefits and Reliability

Performance

- Reduced physical distance to content
- File size reduction: minification of JS/CSS, image compression
- TLS false start: reduces TLS handshake latency for HTTPS connections

Reliability and Redundancy

- Load balancing across edge nodes absorbs traffic spikes
- Intelligent failover: if one PoP goes down, traffic reroutes automatically
- Anycast routing: full data center failure → traffic shifts to next available PoP
- Reduces origin load significantly — origin only serves cache misses

Security

- DDoS mitigation: absorbs attack traffic across global PoPs before it reaches origin
- TLS/SSL termination at the edge: certificates managed by the CDN provider
- Web Application Firewall (WAF) integration available on most cloud CDNs

Cloud CDN Platforms

AWS CloudFront

- AWS's globally distributed CDN with 400+ PoPs (edge locations + regional edge caches)
- Two-tier caching: **Edge Locations** (closest to user) → **Regional Edge Caches** (mid-tier, longer retention) → **Origin**
- Integrates natively with S3 (static assets), EC2/ALB (dynamic), and Lambda@Edge (serverless logic at the edge)
- Supports **cache behaviors**: different cache rules per URL path pattern
- Pricing: pay per GB transferred + per HTTP request (varies by region)

GCP Cloud CDN

- Google's CDN built on top of the **External HTTP(S) Load Balancer**
- Cannot be enabled standalone — requires the load balancer as the entry point
- Cache fill happens from the load balancer's backend (VM instances, GCS buckets, NEGs)
- Leverages Google's global fiber network (same infrastructure that powers Google Search/YouTube)
- Supports **negative caching**: cache non-2xx responses (404, 301, etc.) with configurable TTLs per status code
- Default negative cache TTLs: HTTP 404 → 120 seconds, HTTP 301/308 → 10 minutes, HTTP 302/307 → not cached

Cache Management and Practical Considerations

Cache Invalidation (Purge)

- Forcing the CDN to drop cached content before TTL expires
- Use case: you deployed a new version of a JS file but the CDN is still serving the old one
- AWS CloudFront: **Invalidation** (costs money per path after first 1000/month free)
- GCP Cloud CDN: **Cache invalidation** via API or Console
- Alternative strategy: **cache busting** — append a version hash to file names (app.v2.js), so the CDN treats it as new content automatically

When NOT to Use a CDN

- **When NOT to Use a CDN**
- Highly personalized content (cannot be shared across users at the edge)
- Content that changes faster than any practical TTL (real-time data feeds)
- Internal/private applications not exposed to public internet
- Very low traffic internal tools (overhead not justified by cost)

Common Mistakes

- Forgetting to set Cache-Control headers → CDN uses defaults, often caching things you didn't intend
- Caching POST responses → CDN only caches GET/HEAD by default; POST always goes to origin
- Not invalidating cache after deployment → users see stale content

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS